



Queues

Deep Dive + Real-World Use Cases + Complexity

Java 8+

THIRAVUGAL



Session Structure

1. Queue fundamentals (FIFO)
2. Implementation approaches
3. Circular queue
4. Deque (double-ended queue)
5. Real-world use cases
6. BFS & sliding window
7. Interview-level problems



What is a Queue?

Queue follows:

FIFO (First In, First Out)

Example:

Enqueue: 1, 2, 3

Dequeue → 1



Real World Examples

- Printer queue
- Task scheduling
- Customer service lines
- CPU process scheduling
- Breadth First Search (BFS)



Basic Operations

Operation	Description	Time
enqueue()	Add to rear	$O(1)$
dequeue()	Remove from front	$O(1)$
peek()	View front	$O(1)$
isEmpty()	Check empty	$O(1)$

Space: $O(n)$

Queue Using Array (Basic)

```
class ArrayQueue {
    int[] arr;
    int front = 0, rear = -1, size = 0, capacity;

    ArrayQueue(int capacity) {
        this.capacity = capacity;
        arr = new int[capacity];
    }

    void enqueue(int val) {
        if (size == capacity)
            throw new RuntimeException("Overflow");
        rear++;
        arr[rear] = val;
        size++;
    }

    int dequeue() {
        if (size == 0)
            throw new RuntimeException("Underflow");
        int val = arr[front];
        front++;
        size--;
        return val;
    }
}
```



Circular Queue (Optimized)

```
class CircularQueue {
    int[] arr;
    int front = 0, rear = 0, size = 0, capacity;

    CircularQueue(int capacity) {
        this.capacity = capacity;
        arr = new int[capacity];
    }

    void enqueue(int val) {
        if (size == capacity)
            throw new RuntimeException("Overflow");
        arr[rear] = val;
        rear = (rear + 1) % capacity;
        size++;
    }

    int dequeue() {
        if (size == 0)
            throw new RuntimeException("Underflow");
        int val = arr[front];
        front = (front + 1) % capacity;
        size--;
        return val;
    }
}
```

Queue Using Deque (Best Practice)

```
java.util.Queue<Integer> queue = new java.util.ArrayDeque<>();  
  
queue.offer(10);  
queue.offer(20);  
int front = queue.poll();
```

Why ArrayDeque?

- Faster than LinkedList
- No synchronization overhead



Queue vs Stack

Feature	Stack	Queue
Order	LIFO	FIFO
Insert	Top	Rear
Remove	Top	Front
Use case	Undo	Scheduling



Real World Use Case — Printer Queue

Users send print jobs.

Jobs processed in arrival order.

Each enqueue $\rightarrow O(1)$

Each dequeue $\rightarrow O(1)$



Real World Use Case — Task Scheduler

- Processes wait in queue
- CPU executes first-come task

Used in:

- Operating systems
- Thread pools



Real World Use Case — Breadth First Search (BFS)

BFS uses queue to traverse graph level by level.

Time: $O(V + E)$

Space: $O(V)$

BFS Example (Binary Tree)

```
static void bfs(Node root) {  
    if (root == null) return;  
  
    java.util.Queue<Node> queue = new java.util.ArrayDeque<>();  
    queue.offer(root);  
  
    while (!queue.isEmpty()) {  
        Node current = queue.poll();  
        System.out.print(current.data + " ");  
  
        if (current.left != null) queue.offer(current.left);  
        if (current.right != null) queue.offer(current.right);  
    }  
}
```

Deque (Double-Ended Queue)

Supports:

- Insert at front
- Insert at rear
- Remove from both ends

```
java.util.Deque<Integer> dq = new java.util.ArrayDeque<>();  
dq.offerFirst(1);  
dq.offerLast(2);  
dq.pollFirst();  
dq.pollLast();
```

Sliding Window Maximum (Deque Pattern)

Time: $O(n)$

Space: $O(k)$

```
static int[] maxSlidingWindow(int[] nums, int k) {  
    int n = nums.length;  
    int[] result = new int[n - k + 1];  
    java.util.Deque<Integer> dq = new java.util.ArrayDeque<>();  
  
    for (int i = 0; i < n; i++) {  
        while (!dq.isEmpty() && dq.peekFirst() <= i - k)  
            dq.pollFirst();  
  
        while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i])  
            dq.pollLast();  
    }  
}
```



```
    dq.offerLast(i);

    if (i >= k - 1)
        result[i - k + 1] = nums[dq.peekFirst()];
}
return result;
}
```




Queue Using Two Stacks

Time: Amortized $O(1)$

```
class MyQueue {  
    java.util.Stack<Integer> in = new java.util.Stack<>();  
    java.util.Stack<Integer> out = new java.util.Stack<>();  
  
    void enqueue(int x) {  
        in.push(x);  
    }  
  
    int dequeue() {  
        if (out.isEmpty())  
            while (!in.isEmpty())  
                out.push(in.pop());  
        return out.pop();  
    }  
}
```



Common Mistakes

- Confusing FIFO with LIFO
- Not checking empty before dequeue
- Not using modulo in circular queue
- Using LinkedList unnecessarily
- Forgetting amortized complexity concept



Interview Discussion Points

- Why is ArrayDeque preferred?
- Circular queue advantage?
- Amortized analysis explanation?
- Queue vs Priority Queue difference?



Practice Problems

Easy:

- Implement queue using array
- Reverse first k elements of queue

Medium:

- Sliding window maximum
- First non-repeating character in stream

Advanced:

- Implement LRU cache (queue + hashmap)
- Rotten oranges (BFS grid problem)



Summary

Queues provide:

- FIFO ordering
- $O(1)$ enqueue/dequeue
- Foundation of BFS
- Core to scheduling systems
- Essential for sliding window optimizations

Master:

- Circular queue
- BFS
- Deque usage
- Sliding window pattern